

Production-Ready AWS Infrastructure via Terraform & GitLab CI/CD

An enterprise-grade, GitOps-managed AWS infrastructure codebase designed for high availability, robust security isolation, and automated scalability.

Terraform v1.5+

AWS Cloud

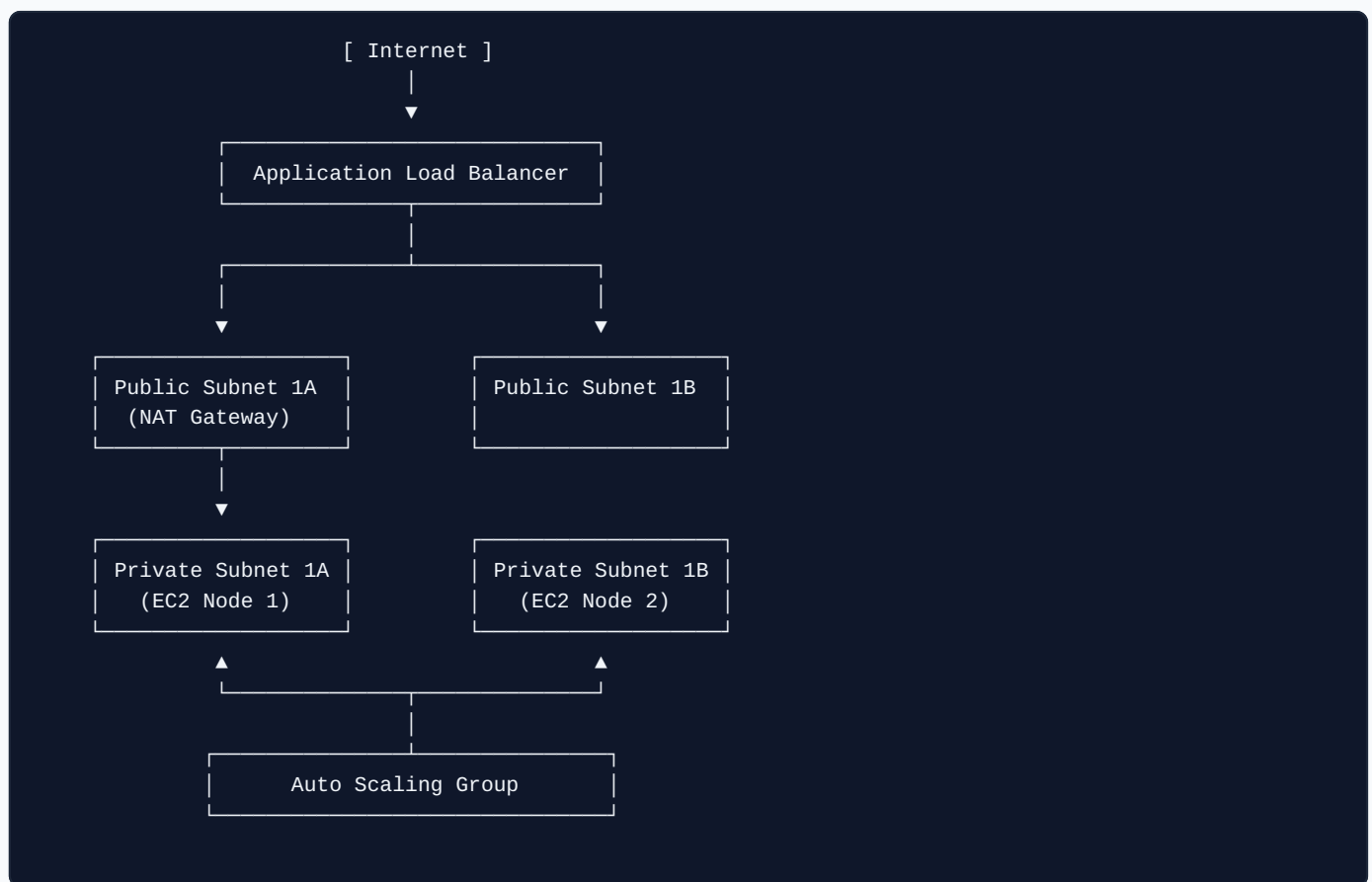
GitLab CI/CD

GitOps

Multi-AZ

1. Architecture Overview

This project provisions a Multi-AZ VPC network architecture hosting an Auto Scaling Group (ASG) behind an Application Load Balancer (ALB), fully automated end-to-end via a GitLab CI/CD pipeline.



Key Architecture Highlights

- **Network Isolation:** Public-facing ALB handles client traffic, forwarding requests strictly to EC2 instances isolated inside private subnets with outbound Internet connectivity provided via NAT Gateway.
- **Least Privilege Security Groups:** EC2 application security groups restrict inbound HTTP access specifically to traffic originating from the Application Load Balancer security group.
- **High Availability & Autoscaling:** Multi-AZ deployment across two Availability Zones with target group health checks (ELB level) and dynamic instance lifecycle management.
- **State Management & Locking:** Remote S3 state storage equipped with server-side encryption and DynamoDB state locking to eliminate race conditions and state corruption.
- **Automated GitOps Pipeline:** Declarative CI/CD pipeline implementing syntax formatting, code validation, speculative planning via stored artifacts, and controlled deployment.

2. Repository Structure

The repository follows standard Terraform modular practices to separate networking, security, and compute concerns:

```
.
├── .gitlab-ci.yml           # GitLab CI/CD pipeline definition
├── main.tf                 # Root module & backend configuration
├── variables.tf            # Global input variables
├── outputs.tf              # Root outputs (e.g., Application URL)
├── terraform.tfvars        # Local environment variable overrides
├── modules/
│   ├── vpc/                # VPC, Subnets, Gateways, Route Tables
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── security_groups/    # Dedicated ALB & EC2 Security Groups
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── compute/            # ALB, Launch Template, ASG, and User-Data
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
```

3. Pipeline Workflow (`.gitlab-ci.yml`)

The infrastructure lifecycle is governed through four structured pipeline stages using HashiCorp's official Terraform Docker runtime image:

1. **validate Stage:** Executes `terraform fmt -check` and `terraform validate` on every branch push to enforce code quality and formatting standard.
2. **plan Stage:** Generates a speculative execution plan (`tfplan`) and persists it as a temporary pipeline artifact (`expire_in: 1 week`).
3. **apply Stage:** Automated execution stage bound to the `main` branch. Consumes the exact artifact output from the `plan` stage to prevent plan drift.
4. **destroy Stage:** Manual pipeline guardrail (`when: manual`) restricted to `main` to allow safe, controlled teardowns of infrastructure environments.

4. Prerequisites & Setup

1. S3 Backend & DynamoDB Lock Provisioning

Before executing the pipeline, initialize remote state infrastructure in your target AWS Account:

- Create an Amazon S3 Bucket (Enable **Bucket Versioning** and **Default SSE-S3 Encryption**).
- Create an AWS DynamoDB Table with a Partition Key named `LockID` of type `String`.
- Update `main.tf` with your backend configuration:

```
terraform {
  backend "s3" {
    bucket      = "YOUR-UNIQUE-TERRAFORM-STATE-BUCKET"
    key         = "gitops-iac/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "YOUR-TERRAFORM-LOCK-TABLE"
```

```
    encrypt      = true
  }
}
```

2. Configure GitLab CI/CD Variables

In your GitLab repository, navigate to **Settings > CI/CD > Variables** and define the following credentials:

Variable Name	Type	Security Options
AWS_ACCESS_KEY_ID	Variable	Protected, Masked
AWS_SECRET_ACCESS_KEY	Variable	Protected, Masked
AWS_DEFAULT_REGION	Variable	Plaintext (e.g., us-east-1)

5. Verification & Deployment

Deployment Steps

1. Commit and push your modifications to the main branch to trigger the pipeline:

```
git add . && git commit -m "feat: deploy multi-AZ web app infrastructure" && git push origin main
```

2. Monitor execution progress under **Build > Pipelines**.

3. Upon successful application, copy the output `application_url` printed in the apply job logs and test instance load balancing in your browser.

6. Core Engineering Skills Demonstrated

- **Infrastructure as Code (IaC):** Enterprise modular Terraform architecture, state locking, dynamic AMI lookups, variable scoping, and clean output abstractions.
- **AWS Cloud Security & Architecture:** High-availability VPC design, NAT Gateways, Multi-AZ ASG, Application Load Balancers, and secure metadata fetching via IMDSv2.
- **DevOps & GitOps:** End-to-end automated pipeline construction, sensitive credential masking, artifact preservation, and deterministic plan execution workflows.